

Exercises week 42

October 13-17, 2025

Date: **Deadline is Friday October 17 at midnight**

Overarching aims of the exercises this week

The aim of the exercises this week is to train the neural network you implemented last week.

To train neural networks, we use gradient descent, since there is no analytical expression for the optimal parameters. This means you will need to compute the gradient of the cost function wrt. the network parameters. And then you will need to implement some gradient method.

You will begin by computing gradients for a network with one layer, then two layers, then any number of layers. Keeping track of the shapes and doing things step by step will be very important this week.

We recommend that you do the exercises this week by editing and running this notebook file, as it includes some checks along the way that you have implemented the neural network correctly, and running small parts of the code at a time will be important for understanding the methods. If you have trouble running a notebook, you can run this notebook in google colab instead(<https://colab.research.google.com/drive/1FfVbN0XlhV-lATRPyGRTtTBnJr3zNuHL#offline=true&sandboxMode=true>), though we recommend that you set up VSCode and your python environment to run code like this locally.

First, some setup code that you will need.

```
In [96]: import autograd.numpy as np # We need to use this numpy wrapper to make automatic differentiation work Later
from autograd import grad, elementwise_grad
from sklearn import datasets
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score

# Defining some activation functions
def ReLU(z):
    return np.where(z > 0, z, 0)

# Derivative of the ReLU function
def ReLU_der(z):
    return np.where(z > 0, 1, 0)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def mse(predict, target):
    return np.mean((predict - target) ** 2)
```

Exercise 1 - Understand the feed forward pass

a) Complete last weeks' exercises if you haven't already (recommended).

Exercise 2 - Gradient with one layer using autograd

For the first few exercises, we will not use batched inputs. Only a single input vector is passed through the layer at a time.

In this exercise you will compute the gradient of a single layer. You only need to change the code in the cells right below an exercise, the rest works out of the box. Feel free to make changes and see how stuff works though!

a) If the weights and bias of a layer has shapes (10, 4) and (10), what will the shapes of the gradients of the cost function wrt. these weights and this bias be?

The gradient will be the same shape as the parameters:

- $\partial C / \partial W$: (10, 4)
- $\partial C / \partial b$: (10,)

b) Complete the feed_forward_one_layer function. It should use the sigmoid activation function. Also define the weight and bias with the correct shapes.

```
In [97]: def feed_forward_one_layer(W, b, x):
    z = np.dot(W, x) + b
    a = sigmoid(z)
    return a

def cost_one_layer(W, b, x, target):
    predict = feed_forward_one_layer(W, b, x)
    return mse(predict, target)

x = np.random.rand(2)
target = np.random.rand(3)
```

```
W = np.random.rand(3, 2)
b = np.random.rand(3)
```

c) Compute the gradient of the cost function wrt. the weigth and bias by running the cell below. You will not need to change anything, just make sure it runs by defining things correctly in the cell above. This code uses the autograd package which uses backpropagation to compute the gradient!

```
In [98]: autograd_one_layer = grad(cost_one_layer, [0, 1])
W_g, b_g = autograd_one_layer(W, b, x, target)
print(W_g, b_g)

[[-0.05003683 -0.02401515]
 [-0.00403704 -0.00193758]
 [-0.00217349 -0.00104317]] [-0.05124312 -0.00413437 -0.00222589]
```

Exercise 3 - Gradient with one layer writing backpropagation by hand

Before you use the gradient you found using autograd, you will have to find the gradient "manually", to better understand how the backpropagation computation works. To do backpropagation "manually", you will need to write out expressions for many derivatives along the computation.

We want to find the gradient of the cost function wrt. the weight and bias. This is quite hard to do directly, so we instead use the chain rule to combine multiple derivatives which are easier to compute.

$$\frac{dC}{dW} = \frac{dC}{da} \frac{da}{dz} \frac{dz}{dW}$$
$$\frac{dC}{db} = \frac{dC}{da} \frac{da}{dz} \frac{dz}{db}$$

a) Which intermediary results can be reused between the two expressions?

We can reuse the shared backprop pieces:

- dC/da and da/dz are common to both.
- Then compute the error term once: $\sigma = dC/dz = (dC/da) \cdot (da/dz)$, and reuse it for both gradients.

b) What is the derivative of the cost wrt. the final activation? You can use the autograd calculation to make sure you get the correct result. Remember that we compute the mean in mse.

For MSE defined as $\text{mean}((\text{predict} - \text{target})^2)$, the derivative wrt the final activation a is: $dC/da = (2 / N) \cdot (a - \text{target})$

```
In [99]: z = W @ x + b
a = sigmoid(z)

def mse_der(a, target):
    return (2 / target.size) * (a - target)

print(mse_der(a, target))

cost_autograd = grad(mse, 0)
print(cost_autograd(a, target))
assert np.allclose(mse_der(a, target), cost_autograd(a, target))

[-0.2230192 -0.01696527 -0.01110467]
[-0.2230192 -0.01696527 -0.01110467]
```

c) What is the expression for the derivative of the sigmoid activation function? You can use the autograd calculation to make sure you get the correct result.

For $\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$, the derivative is: $ds/dz = \text{sigmoid}(z) \cdot (1 - \text{sigmoid}(z))$ If you already computed $a = \text{sigmoid}(z)$, then $da/dz = a \cdot (1 - a)$.

```
In [100... def sigmoid_der(z):
    a = sigmoid(z)
    return a * (1 - a)

print(sigmoid_der(z))

sigmoid_autograd = elementwise_grad(sigmoid, 0)
print(sigmoid_autograd(z))
assert np.allclose(sigmoid_der(z), sigmoid_autograd(z))

[0.22976998 0.24369602 0.20044631]
[0.22976998 0.24369602 0.20044631]
```

d) Using the two derivatives you just computed, compute this intermetidary gradient you will use later:

$$\frac{dC}{dz} = \frac{dC}{da} \frac{da}{dz}$$

```
In [101... dC_da = (2 / target.size) * (a - target)
dC_dz = dC_da * sigmoid_der(z)
```

e) What is the derivative of the intermediary z wrt. the weight and bias? What should the shapes be? The one for the weights is a little tricky, it can be easier to play around in the next exercise first. You can also try computing it with autograd to get a hint.

Let $z = W x + b$ with $W \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, $b \in \mathbb{R}^m$, $z \in \mathbb{R}^m$. Component-wise: $z_i = \sum_j W_{[ij]} x_j + b_i$.

- dC/dW: (m, n) — same as W.
- dC/db: (m,) — same as b.

f) Now combine the expressions you have worked with so far to compute the gradients! Note that you always need to do a feed forward pass while saving the zs and as before you do backpropagation, as they are used in the derivative expressions

In [102...

```

dC_da = (2 / target.size) * (a - target)
dC_dz = dC_da * sigmoid_der(z)

# Gradients for one sample
dC_dW = np.outer(dC_dz, x)
dC_db = dC_dz

print(dC_dW.shape, dC_db.shape)
print(dC_dW, dC_db)

(3, 2) (3,)
[[-0.05003683 -0.02401515]
 [-0.00403704 -0.00193758]
 [-0.00217349 -0.00104317]] [-0.05124312 -0.00413437 -0.00222589]

```

You should get the same results as with autograd.

In [103...

```

W_g, b_g = autograd_one_layer(W, b, x, target)
print(W_g, b_g)
assert np.allclose(dC_dW, W_g)
assert np.allclose(dC_db, b_g)

[[-0.05003683 -0.02401515]
 [-0.00403704 -0.00193758]
 [-0.00217349 -0.00104317]] [-0.05124312 -0.00413437 -0.00222589]

```

Exercise 4 - Gradient with two layers writing backpropagation by hand

Now that you have implemented backpropagation for one layer, you have found most of the expressions you will need for more layers. Let's move up to two layers.

In [104...

```

x = np.random.rand(2)
target = np.random.rand(4)

W1 = np.random.rand(3, 2)
b1 = np.random.rand(3)

W2 = np.random.rand(4, 3)
b2 = np.random.rand(4)

layers = [(W1, b1), (W2, b2)]

```

In [105...

```

z1 = W1 @ x + b1
a1 = sigmoid(z1)
z2 = W2 @ a1 + b2
a2 = sigmoid(z2)

```

We begin by computing the gradients of the last layer, as the gradients must be propagated backwards from the end.

a) Compute the gradients of the last layer, just like you did the single layer in the previous exercise.

In [106...

```

dC_da2 = (2 / target.size) * (a2 - target)
dC_dz2 = dC_da2 * sigmoid_der(z2)
dC_dW2 = np.outer(dC_dz2, a1) # use a1 input to Layer 2
dC_db2 = dC_dz2
print(dC_dW2.shape, dC_db2.shape)

(4, 3) (4,)

```

To find the derivative of the cost wrt. the activation of the first layer, we need a new expression, the one furthest to the right in the following.

$$\frac{dC}{da_1} = \frac{dC}{dz_2} \frac{dz_2}{da_1}$$

b) What is the derivative of the second layer intermetiate wrt. the first layer activation? (First recall how you compute z_2)

$$\frac{dz_2}{da_1}$$

Since $z_2 = W_2a_1 + b_2$, the Jacobian is the weight matrix:

- $\frac{dz_2}{da_1} = W_2$ (shape: output_dim × hidden_dim).

This is used in backprop as $\frac{dC}{da_1} = W_2^T \frac{dC}{dz_2}$.

In [107...

```

dC_da1 = W2.T @ dC_dz2
print(dC_da1.shape)

(3,)

```

c) Use this expression, together with expressions which are equivalent to ones for the last layer to compute all the derivatives of the first layer.

$$\frac{dC}{dW_1} = \frac{dC}{da_1} \frac{da_1}{dz_1} \frac{dz_1}{dW_1}$$

$$\frac{dC}{db_1} = \frac{dC}{da_1} \frac{da_1}{dz_1} \frac{dz_1}{db_1}$$

```
In [108... dC_da1 = W2.T @ dC_dz2
dC_dz1 = dC_da1 * sigmoid_der(z1)
dC_dw1 = np.outer(dC_dz1, x)
dC_db1 = dC_dz1
```

```
In [109... print(dC_dw1, dC_db1)
print(dC_dw2, dC_db2)
```

```
[0.00465301 0.00217942]
[0.00870157 0.00407571]
[0.00274016 0.00128346]] [0.00821214 0.01535748 0.00483613]
[[ 0.01590261  0.01260715  0.01632696]
 [ 0.03045351  0.02414271  0.03126615]
 [ 0.01297365  0.01028515  0.01331984]
 [-0.00276361 -0.00219091 -0.00283735]] [ 0.02208896  0.04230037  0.01802058 -0.00383869]
```

d) Make sure you got the same gradient as the following code which uses autograd to do backpropagation.

```
In [110... def feed_forward_two_layers(layers, x):
    w1, b1 = layers[0]
    z1 = W1 @ x + b1
    a1 = sigmoid(z1)

    w2, b2 = layers[1]
    z2 = W2 @ a1 + b2
    a2 = sigmoid(z2)

    return a2
```

```
In [111... def cost_two_layers(layers, x, target):
    predict = feed_forward_two_layers(layers, x)
    return mse(predict, target)

grad_two_layers = grad(cost_two_layers, 0)
grad_two_layers(layers, x, target)
```

```
Out[111... [(array([[0.00465301, 0.00217942],
          [0.00870157, 0.00407571],
          [0.00274016, 0.00128346]]),
  array([0.00821214, 0.01535748, 0.00483613])),
 (array([[ 0.01590261,  0.01260715,  0.01632696],
          [ 0.03045351,  0.02414271,  0.03126615],
          [ 0.01297365,  0.01028515,  0.01331984],
          [-0.00276361, -0.00219091, -0.00283735]]),
  array([ 0.02208896,  0.04230037,  0.01802058, -0.00383869])))]
```

e) How would you use the gradient from this layer to compute the gradient of an even earlier layer? Would the expressions be any different?

You reuse the exact same backprop pattern recursively. Given the error of layer l as $\delta_l = \frac{dC}{dz_l}$, propagate to the previous layer $l - 1$

The expressions are not different. It's the same chain-rule steps repeated for each earlier layer, only with the appropriate layer's weights and activation derivative.

Exercise 5 - Gradient with any number of layers writing backpropagation by hand

Well done on getting this far! Now it's time to compute the gradient with any number of layers.

First, some code from the general neural network code from last week. Note that we are still sending in one input vector at a time. We will change it to use batched inputs later.

```
In [112... def create_layers(network_input_size, layer_output_sizes):
    layers = []

    i_size = network_input_size
    for layer_output_size in layer_output_sizes:
        W = np.random.randn(layer_output_size, i_size)
        b = np.random.randn(layer_output_size)
        layers.append((W, b))

        i_size = layer_output_size
    return layers

def feed_forward(input, layers, activation_funcs):
    a = input
    for (W, b), activation_func in zip(layers, activation_funcs):
        z = W @ a + b
        a = activation_func(z)
    return a

def cost(layers, input, activation_funcs, target):
    predict = feed_forward(input, layers, activation_funcs)
    return mse(predict, target)
```

You might have already have noticed a very important detail in backpropagation: You need the values from the forward pass to compute all the gradients! The feed forward method above is great for efficiency and for using autograd, as it only cares about computing the final

output, but now we need to also save the results along the way.

Here is a function which does that for you.

```
In [113... def feed_forward_saver(input, layers, activation_funcs):
    layer_inputs = []
    zs = []
    a = input
    for (W, b), activation_func in zip(layers, activation_funcs):
        layer_inputs.append(a)
        z = W @ a + b
        a = activation_func(z)

        zs.append(z)

    return layer_inputs, zs, a
```

a) Now, complete the backpropagation function so that it returns the gradient of the cost function wrt. all the weights and biases. Use the autograd calculation below to make sure you get the correct answer.

```
In [114... def backpropagation(
    input, layers, activation_funcs, target, activation_ders, cost_der=mse_der
):
    layer_inputs, zs, predict = feed_forward_saver(input, layers, activation_funcs)

    layer_grads = [() for layer in layers]

    # We Loop over the Layers, from the last to the first
    for i in reversed(range(len(layers))):
        layer_input, z, activation_der = layer_inputs[i], zs[i], activation_ders[i]

        if i == len(layers) - 1:
            # For last Layer we use cost derivative as dC_da(L) can be computed directly
            dC_da = cost_der(predict, target)
        else:
            # For other Layers we build on next Layer's z derivative: dC/da(i) = W(i+1)^T @ dC/dz(i+1)
            (W_next, _b_next) = layers[i + 1]
            dC_da = W_next.T @ dC_dz

            # Local backprop through activation and affine
            dC_dz = dC_da * activation_der(z)
            dC_dW = np.outer(dC_dz, layer_input)
            dC_db = dC_dz

        layer_grads[i] = (dC_dW, dC_db)

    return layer_grads
```

```
In [115... network_input_size = 2
layer_output_sizes = [3, 4]
activation_funcs = [sigmoid, ReLU]
activation_ders = [sigmoid_der, ReLU_der]

layers = create_layers(network_input_size, layer_output_sizes)

x = np.random.rand(network_input_size)
target = np.random.rand(4)
```

```
In [116... layer_grads = backpropagation(x, layers, activation_funcs, target, activation_ders)
print(layer_grads)

[(array([[ -0.06010538, -0.12490722],
        [ 0.14998254,  0.31168425],
        [-0.05235546, -0.10880182]]), array([ -0.13932038,  0.34764981, -0.12135657])), (array([[ 0.65254806,  0.68450693,
0.64042198],
        [ 0.50831332,  0.53320822,  0.4988675 ],
        [-0.          , -0.          , -0.          ],
        [-0.18342024, -0.19240334, -0.18001181]]), array([ 0.8585453 ,  0.66877834, -0.          , -0.24132259]))]
```

```
In [117... cost_grad = grad(cost, 0)
autograd_layer_grads = cost_grad(layers, x, [sigmoid, ReLU], target)

# Compare
for i, ((dW_m, db_m), (dW_a, db_a)) in enumerate(zip(layer_grads, autograd_layer_grads)):
    print(f'Layer {i}: allclose(dW) ->', np.allclose(dW_m, dW_a), ', allclose(db) ->', np.allclose(db_m, db_a))
    assert np.allclose(dW_m, dW_a)
    assert np.allclose(db_m, db_a)
print('Backpropagation gradients match autograd for all layers.')
```

Layer 0: allclose(dW) -> True , allclose(db) -> True
Layer 1: allclose(dW) -> True , allclose(db) -> True
Backpropagation gradients match autograd for all layers.

Exercise 6 - Batched inputs

Make new versions of all the functions in exercise 5 which now take batched inputs instead. See last weeks exercise 5 for details on how to batch inputs to neural networks. You will also need to update the backpropogation function.

```
In [118... # Batched versions of forward pass, saver, cost, and backpropagation

def feed_forward_batch(inputs, layers, activation_funcs):
    A = inputs
    for (W, b), activation_func in zip(layers, activation_funcs):
        Z = A @ W.T + b # (B, out_dim)
        A = activation_func(Z)
    return A
```

```

def cost_batch(layers, inputs, activation_funcs, targets):
    predict = feed_forward_batch(inputs, layers, activation_funcs)
    return np.mean((predict - targets) ** 2)

def feed_forward_saver_batch(inputs, layers, activation_funcs):
    layer_inputs = []
    zs = []
    A = inputs
    for (W, b), activation_func in zip(layers, activation_funcs):
        layer_inputs.append(A)
        Z = A @ W.T + b
        A = activation_func(Z)
        zs.append(Z)
    return layer_inputs, zs, A

def backpropagation_batch(inputs, layers, activation_funcs, targets, activation_ders, cost_der=mse_der):
    layer_inputs, zs, predict = feed_forward_saver_batch(inputs, layers, activation_funcs)
    layer_grads = [None for _ in layers]

    dZ_next = None
    for i in reversed(range(len(layers))):
        A_prev = layer_inputs[i]
        Z = zs[i]
        act_der = activation_ders[i]

        if i == len(layers) - 1:
            # Head Layer
            dA = cost_der(predict, targets)
        else:
            # Hidden Layers
            W_next, _ = layers[i + 1]
            dA = dZ_next @ W_next

        dZ = dA * act_der(Z)
        dW = dZ.T @ A_prev
        db = dZ.sum(axis=0)

        layer_grads[i] = (dW, db)
        dZ_next = dZ

    return layer_grads

# check vs autograd on a random mini-batch
network_input_size = 2
layer_output_sizes = [3, 4]
activation_funcs = [sigmoid, ReLU]
activation_ders = [sigmoid_der, ReLU_der]

layers = create_layers(network_input_size, layer_output_sizes)

B = 5
inputs = np.random.rand(B, network_input_size)
targets = np.random.rand(B, layer_output_sizes[-1])

manual_grads = backpropagation_batch(inputs, layers, activation_funcs, targets, activation_ders)

cost_grad_batched = grad(cost_batch, 0)
autograd_grads = cost_grad_batched(layers, inputs, activation_funcs, targets)

for i, ((dW_m, db_m), (dW_a, db_a)) in enumerate(zip(manual_grads, autograd_grads)):
    print(f'Layer {i}: allclose(dW) ->', np.allclose(dW_m, dW_a), ', allclose(db) ->', np.allclose(db_m, db_a))
    assert np.allclose(dW_m, dW_a)
    assert np.allclose(db_m, db_a)
print('Batched backpropagation gradients match autograd for all layers.')

```

Layer 0: allclose(dW) -> True , allclose(db) -> True
 Layer 1: allclose(dW) -> True , allclose(db) -> True
 Batched backpropagation gradients match autograd for all layers.

Exercise 7 - Training

a) Complete exercise 6 and 7 from last week, but use your own backpropogation implementation to compute the gradient.

- IMPORTANT: Do not implement the derivative terms for softmax and cross-entropy separately, it will be very hard!
- Instead, use the fact that the derivatives multiplied together simplify to **prediction - target** (see [source1](#), [source2](#))

b) Use stochastic gradient descent with momentum when you train your network.

Exercise 8 (Optional) - Object orientation

Passing in the layers, activations functions, activation derivatives and cost derivatives into the functions each time leads to code which is easy to understand in isolation, but messier when used in a larger context with data splitting, data scaling, gradient methods and so forth. Creating an object which stores these values can lead to code which is much easier to use.

a) Write a neural network class. You are free to implement it how you see fit, though we strongly recommend to not save any input or output values as class attributes, nor let the neural network class handle gradient methods internally. Gradient methods should be handled outside, by performing general operations on the layer_grads list using functions or classes separate to the neural network.

We provide here a skeleton structure which should get you started.

In [119...

```
class NeuralNetwork:
    def __init__(
        self,
        network_input_size,
        layer_output_sizes,
        activation_funcs,
        activation_ders,
        cost_fun,
        cost_der,
    ):
        # Store configuration
        self.input_size = network_input_size
        self.layer_output_sizes = layer_output_sizes
        self.activation_funcs = activation_funcs
        self.activation_ders = activation_ders
        self.cost_fun = cost_fun
        self.cost_der = cost_der

        # Initialize layers (weights, biases)
        self.layers = create_layers(network_input_size, layer_output_sizes)

    def predict(self, inputs):
        # Simple feed forward pass (handles single or batched inputs)
        if inputs.ndim == 1:
            return feed_forward(inputs, self.layers, self.activation_funcs)
        return feed_forward_batch(inputs, self.layers, self.activation_funcs)

    def cost(self, inputs, targets):
        predict = self.predict(inputs)
        return self.cost_fun(predict, targets)

    def _feed_forward_saver(self, inputs):
        # Utility to save intermediates; not stored on self (returned instead)
        if inputs.ndim == 1:
            return feed_forward_saver(inputs, self.layers, self.activation_funcs)
        return feed_forward_saver_batch(inputs, self.layers, self.activation_funcs)

    def compute_gradient(self, inputs, targets):
        # Delegates to the non-OOP backprop helpers to keep logic centralized
        if inputs.ndim == 1:
            return backpropagation(inputs, self.layers, self.activation_funcs, targets, self.activation_ders, self.cost_der)
        return backpropagation_batch(inputs, self.layers, self.activation_funcs, targets, self.activation_ders, self.cost_der)

    def update_weights(self, layer_grads):
        # Apply parameter deltas (dW, db) to current layers.
        # Expectation: Layer_grads already include learning-rate scaling and any optimizer logic.
        new_layers = []
        for (W, b), (dW, db) in zip(self.layers, layer_grads):
            new_layers.append((W + dW, b + db))
        self.layers = new_layers

    # These last two methods are not needed in the project, but they can be nice to have! The first one has a Layers parameter
    def autograd_compliant_predict(self, layers, inputs):
        # Forward pass using provided layers (not self.layers) for autograd
        A = inputs
        if inputs.ndim == 1:
            for (W, b), activation_func in zip(layers, self.activation_funcs):
                Z = W @ A + b
                A = activation_func(Z)
            return A
        # Batched
        for (W, b), activation_func in zip(layers, self.activation_funcs):
            Z = A @ W.T + b
            A = activation_func(Z)
        return A

    def autograd_gradient(self, inputs, targets):
        # Returns autograd-computed gradients wrt. self.layers
        def local_cost(layers):
            preds = self.autograd_compliant_predict(layers, inputs)
            return self.cost_fun(preds, targets)
        g_layers = grad(local_cost, 0)
        return g_layers(self.layers)
```

In [120...

```
# Quick sanity check for the NeuralNetwork class vs autograd (batched)
np.random.seed(0)
net = NeuralNetwork(
    network_input_size=2,
    layer_output_sizes=[3, 4],
    activation_funcs=[sigmoid, ReLU],
    activation_ders=[sigmoid_der, ReLU_der],
    cost_fun=mse,
    cost_der=mse_der,
)

B = 6
X = np.random.rand(B, 2)
Y = np.random.rand(B, 4)

manual_grads = net.compute_gradient(X, Y)
auto_grads = net.autograd_gradient(X, Y)

for i, ((dW_m, db_m), (dW_a, db_a)) in enumerate(zip(manual_grads, auto_grads)):
    print(f'Layer {i}: dW match ->', np.allclose(dW_m, dW_a), ', db match ->', np.allclose(db_m, db_a))
    assert np.allclose(dW_m, dW_a)
    assert np.allclose(db_m, db_a)
print('NeuralNetwork.compute_gradient matches autograd for all layers (batched).')
```

Layer 0: dW match -> True , db match -> True
Layer 1: dW match -> True , db match -> True
NeuralNetwork.compute_gradient matches autograd for all layers (batched).

